School of Computer Science and Artificial Intelligence

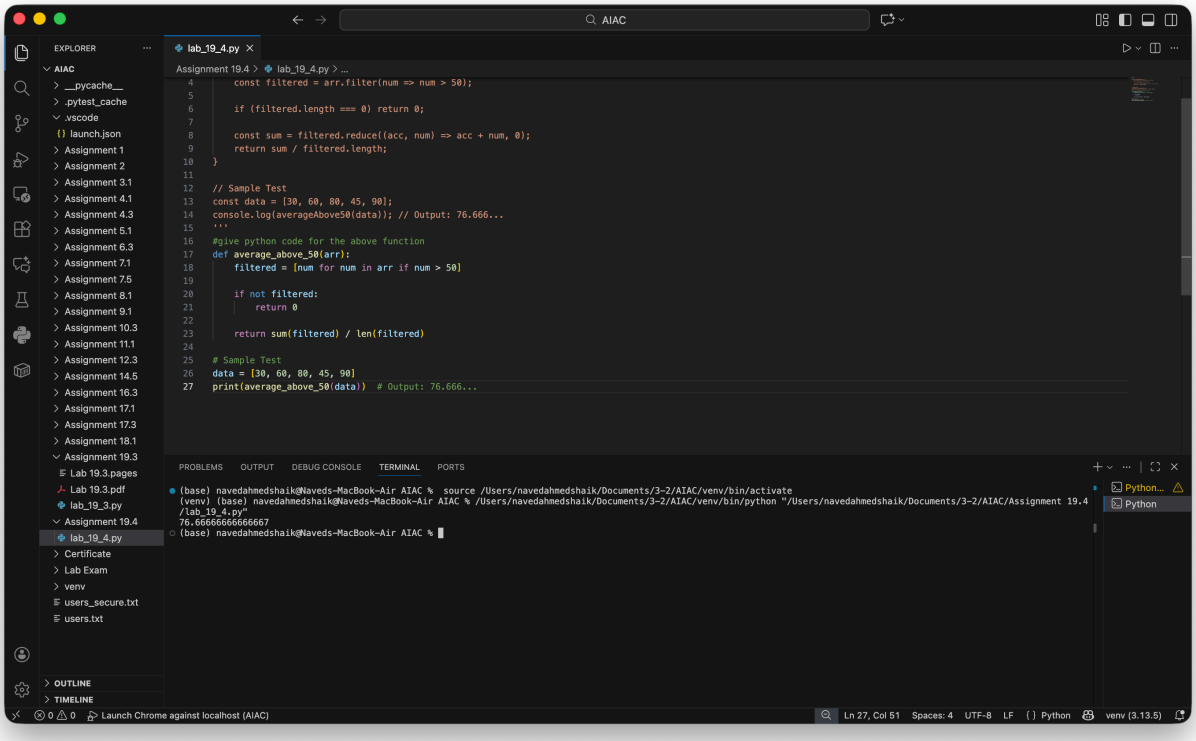
Lab Assignment - 19.4

Course Name : AI Assistant Coding
Name of the Student : Shaik Naved Ahmed
Registration No : 2303A54053
Batch No : 47-B

Task 1 – JavaScript to Python (Array Processing Logic)
Scenario
A frontend application written in JavaScript processes user scores. The backend team wants the same logic implemented in Python for analytics processing.
Task
Translate a JavaScript function that:
- **Takes an array of numbers**
- **Filters values greater than 50**
- **Returns the average of filtered values**

Instructions
- **Prompt AI to convert JavaScript array methods (filter, reduce) into equivalent Python logic.**
- **Ensure list comprehension or Pythonic style is used.**
- **Compare outputs of both implementations with sample input.**
- **Document differences in handling arrays between JS and Python.**

Expected Output
- **Equivalent Python function.**
- **Same output for identical inputs.**
- **Clean and Pythonic implementation.**



Feature	JavaScript	Python
Array Type	Dynamic Array ([])	List ([])
Filter	arr.filter()	filter() or list comprehension
Reduce	arr.reduce()	sum() or functools.reduce()
Empty Check	length === 0	if not list
Syntax Style	Functional-heavy	More Pythonic with comprehensions

Task 2 – Python OOP to C++ Class Translation

Scenario

A data modeling module is written in Python using OOP. The system is being migrated to a high-performance C++ environment.

Task

Translate a Python class that:

- **Contains private attributes**
- **Uses getter/setter methods**
- **Includes a method that calculates a derived value**

Instructions

- **Convert the Python class into a C++ class.**
- **Ensure:**

o Proper access modifiers

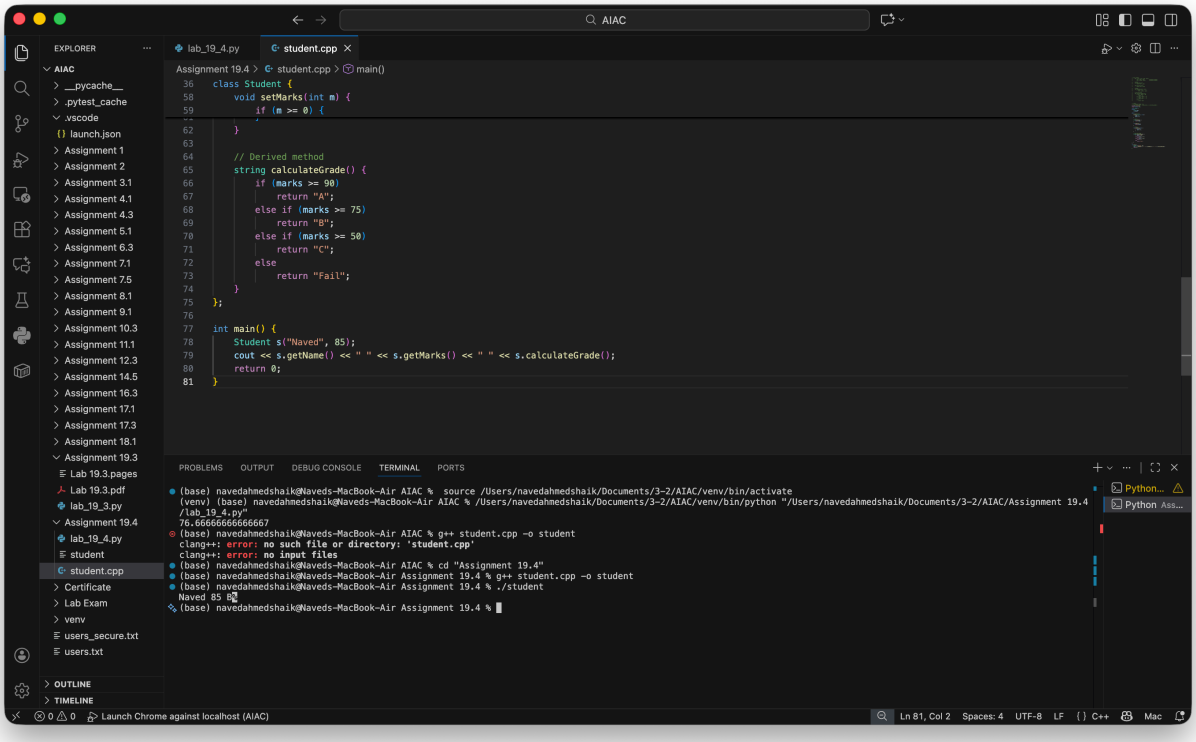
o Constructor handling

o Correct data types

- **Compile and test the C++ version.**
- **Compare how encapsulation differs between Python and C++.**

Expected Output

- **Fully working C++ class equivalent to Python version.**
- **Proper use of access specifiers (private, public).**



Feature	Python	C++
Private Members	__var (name mangling)	private: keyword
Enforcement	Weak (can still access)	Strong (strictly enforced)
Getter/Setter	Optional	Common practice
Access Control	Flexible	Strict compile-time control
Data Types	Dynamic	Must define (int, string)

Task 3 – REST API Call Translation: Python to JavaScript

Scenario

A backend developer wrote an API request using Python requests. The frontend team needs the same functionality in JavaScript (Fetch API).

Task

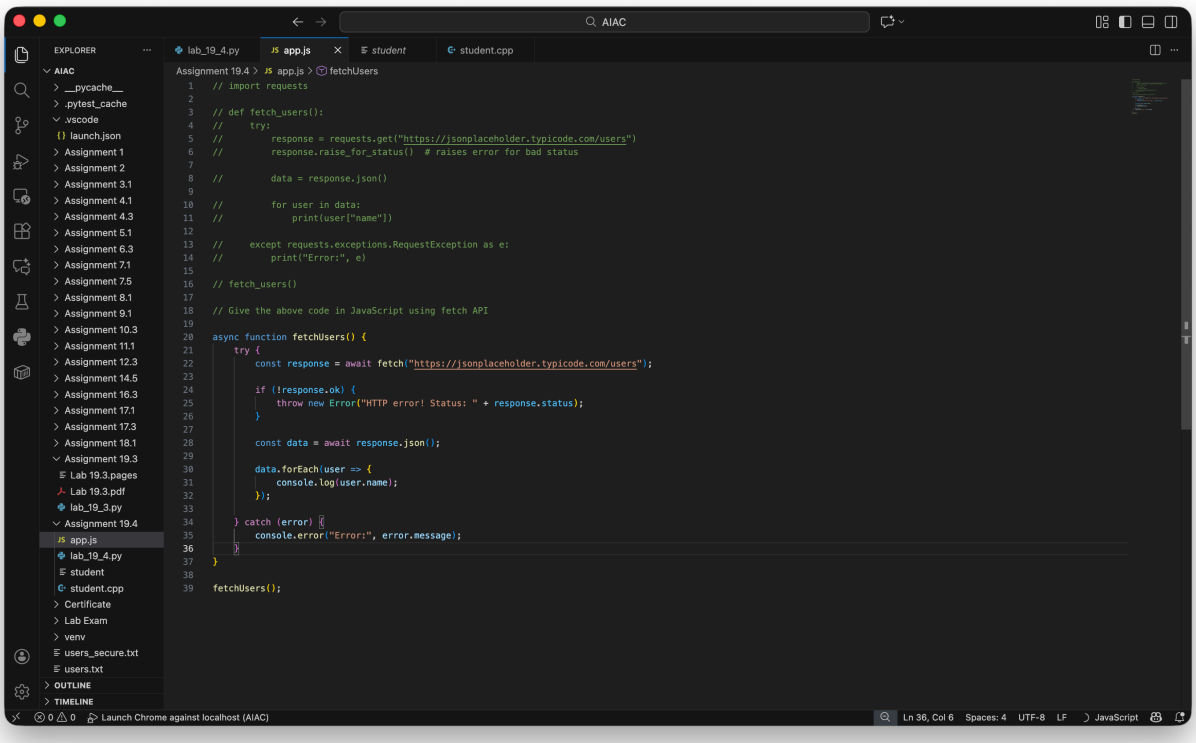
Translate Python API request code into JavaScript.

Instructions

- **Convert:**
  - **GET request logic**
  - **JSON parsing**
  - **Error handling (try-except → try-catch)**
- **Ensure proper async/await usage in JavaScript.**
- **Compare differences in error handling mechanisms.**

Expected Output

- **Equivalent JavaScript code using Fetch API.**
- **Proper async handling.**
- **Similar output structure.**



Feature	Python (requests)	JavaScript (fetch)
Request	requests.get()	fetch()
Async Handling	Blocking (sync)	Non-blocking (async/await)
JSON Parsing	response.json()	await response.json()
Error Handling	try-except	try-catch
HTTP Errors	raise_for_status()	Manual check (response.ok)
Execution	Direct	Needs async function

Task 4 – SQL Aggregation to MongoDB Query Translation
Scenario
A company is migrating from relational databases (SQL) to MongoDB (NoSQL).
Task
Translate an SQL aggregation query into a MongoDB aggregation pipeline.
Example Logic:
• Group records by category
• Calculate average price
• Filter groups where average price > threshold
Instructions
• Provide SQL query to AI.
• Generate equivalent MongoDB query.
• Explain differences between relational and document-based querying.
• Validate results conceptually.
Expected Output
• Working MongoDB aggregation query.
• Clear understanding of grouping and aggregation differences.

STEP 1: Install MongoDB

Option A (Recommended – Mac using Homebrew)

```
brew tap mongodb/brew
brew install mongodb-community
```

Start MongoDB

```
brew services start mongodb-community
```

STEP 2: Open Mongo Shell

```
mongosh
```

If it opens → success ✔️

STEP 3: Create Database

```
use companyDB
```

STEP 4: Create Collection & Insert Data

```
db.products.insertMany([
  { category: "Electronics", price: 120 },
  { category: "Electronics", price: 80 },
  { category: "Clothing", price: 50 },
  { category: "Clothing", price: 60 },
  { category: "Furniture", price: 200 },
  { category: "Furniture", price: 300 }
])
```

STEP 5: Verify Data

```
db.products.find()
```

STEP 6: SQL Query (Reference)

```
SELECT category, AVG(price)
FROM products
GROUP BY category
HAVING AVG(price) > 100;
```

STEP 7: Run MongoDB Query

```
db.products.aggregate([
  {
    $group: {
      _id: "$category",
      avg_price: { $avg: "$price" }
    }
  },
  {
    $match: {
      avg_price: { $gt: 100 }
    }
  }
])
```

STEP 8: Expected Output

```
[
  { _id: 'Furniture', avg_price: 250 }
]
```

Only categories with avg > 100 appear ✅

STEP 9: Understand What Happened

- $group → groups by category
- $avg → calculates average
- $match → filters results (like HAVING)

STEP 10: Run in VS Code (Alternative)

Install Extension:

- MongoDB for VS Code

Then:

1. Open VS Code
2. Click MongoDB icon
3. Connect to mongodb://localhost:27017
4. Run same query

Task 5 – Real-Time Application: Algorithm Translation Across Three Languages

Scenario

A tech company maintains codebases in multiple languages (Python, Java, and Go). A common algorithm must be implemented consistently across systems.

Task

Choose an algorithm (e.g., binary search, palindrome check, prime number checker) and translate it across:

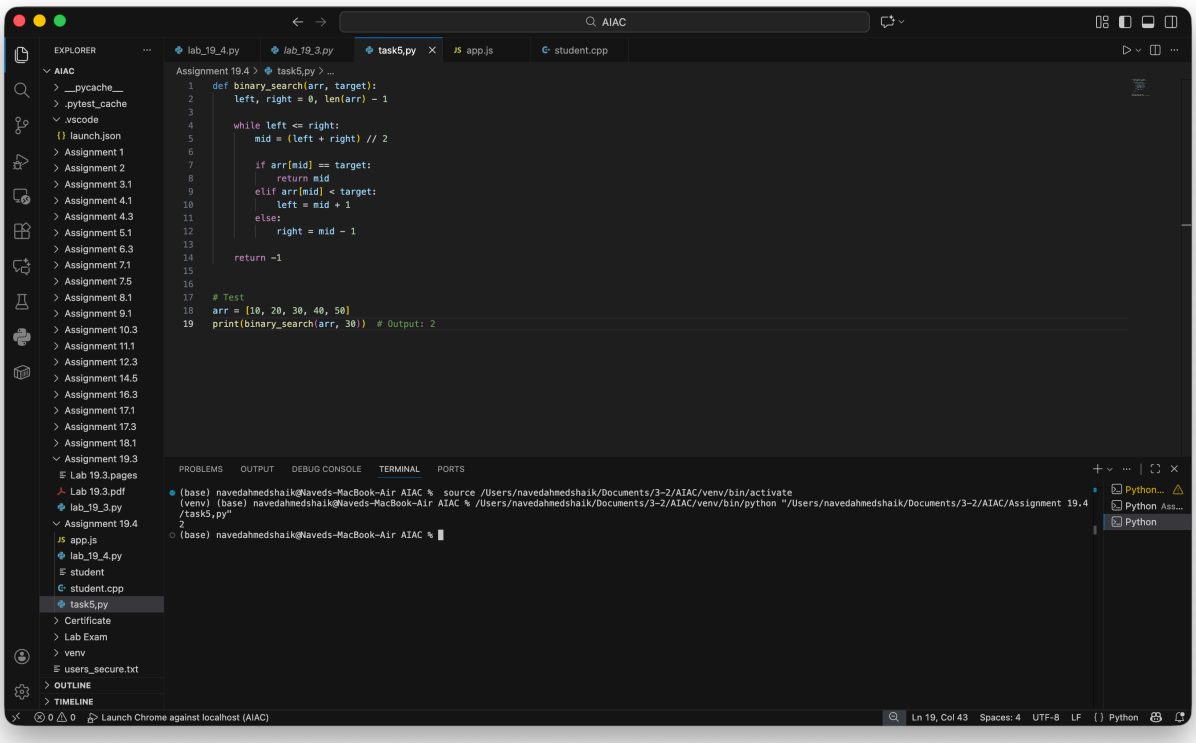
• Python
• Java
• One additional language (Go / C# / JavaScript)

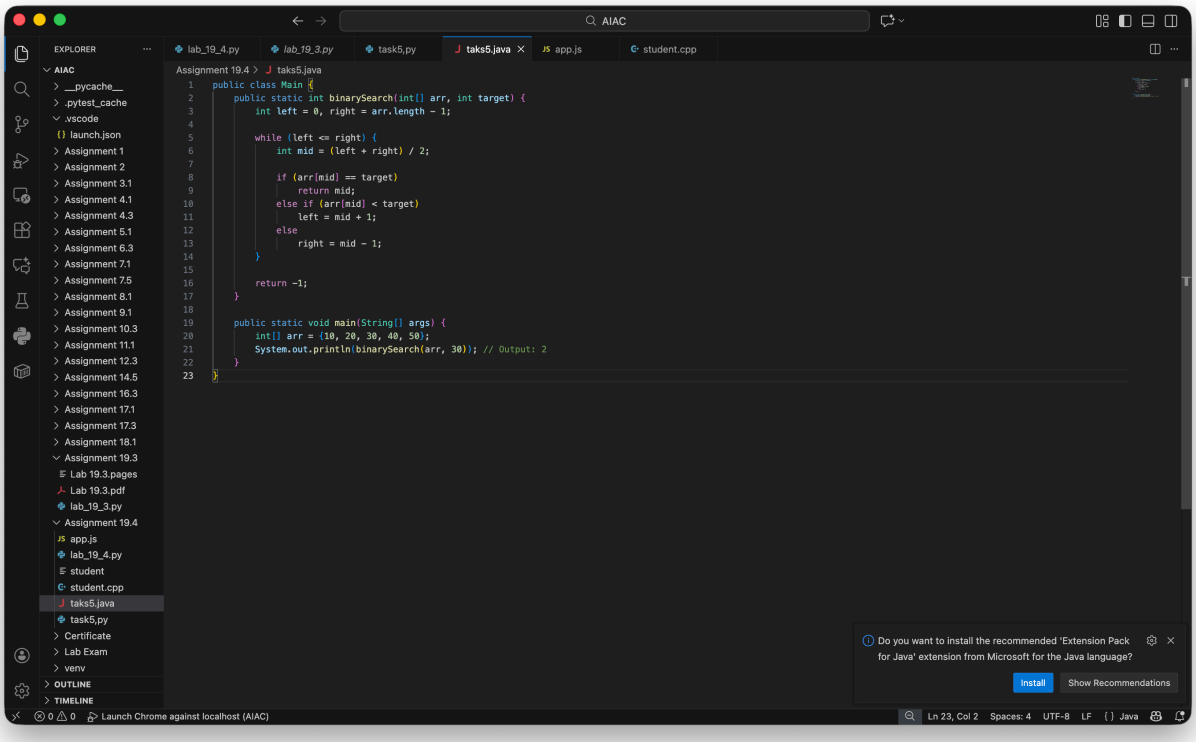
Instructions

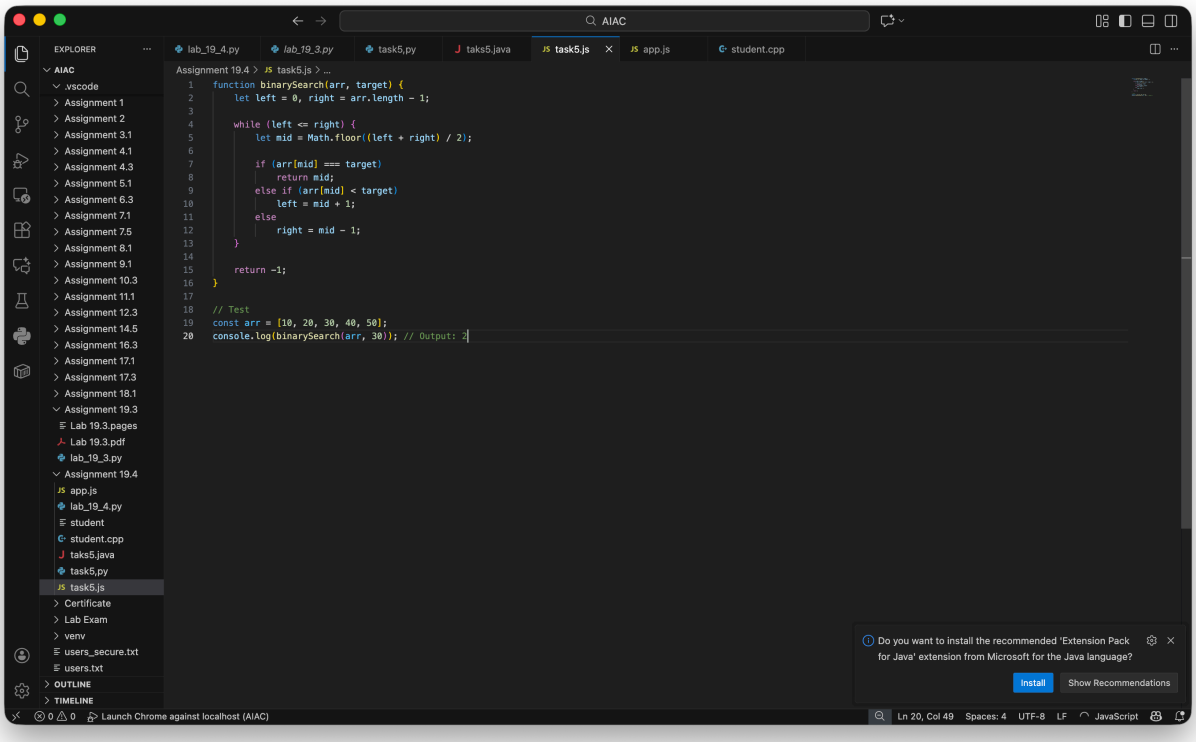
• Ensure logic remains identical across languages.
• Handle:
o Data types
o Loop differences
o Input/output handling
• Test each version with same input values.
• Briefly document translation challenges.

Expected Output

• Three equivalent working implementations.
• Clear logical consistency.
• Understanding of syntactic vs structural differences.







Feature	Python	Java	JavaScript
Data Types	Dynamic	Static (int[])	Dynamic
Division	//	/	Math.floor()
Syntax	Simple	Verbose	Moderate
Entry Point	Direct run	main() required	Direct run